

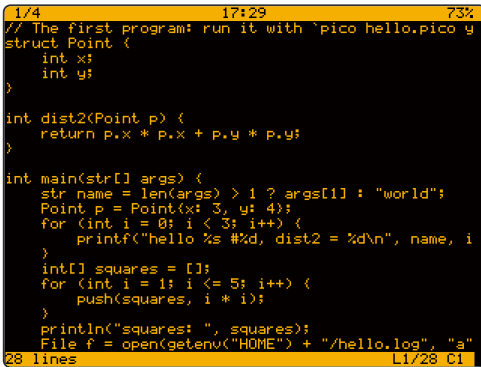
Type along. Every example here runs as it is: save it with `~/pico`, to read and change. `man pico` is the short reference on the machine.

16	Programs to keep
15	Reference
14	When it goes wrong
13	The internet
12	Digital rain
11	A game: snake
10	The terminal
PART 2: APPS	
8	Text and structs
7	Functions and lists
6	Choosing: switch and enum
5	Deciding and repeating
4	Values and variables
3	The first program
PART 1: THE LANGUAGE	
Contents	

PART 1 OF 2 · THE LANGUAGE

pico.

the language of PocketType



C-like and typed, **safe**: a mistake stops the program at its line, never the machine.

Part 1 the language · **Part 2** apps: the terminal, games, the internet

1.0 beta 4

```
$ pico hello.pico andre
hello, andre

}
return 0;
println("hello, ", who);
str who = len(args) > 1 ? args[1] : "you";
int main(str[] args) {
    name:
    Every program has a main. What it returns is its exit status: 0
    means all went well, main can have the words typed after the
    a program of it, ./hello, that starts faster
    pico compiles and runs in one go, pico hello.pico makes
    hello, pocket
    $ pico hello.pico
```

Write it with edit `hello.pico`, then `Fn S` to save and `Fn Q` to leave, and run it:

```
int main() {
    println("hello, pocket");
    return 0;
}
```

The first program

Text and structs

```
str line = " the quick brown fox ";
str[] words = split(trim(line));
println(join(words, "-")); // the-quick-brown-fox
println(upper("hey"), find("hello", "ll"));
```

Also `substr`, `replace`, `contains`, `starts_with`, `repeat`, `chars`. `s[i]` is the byte at `i`, as a number.

STRUCTS

```
struct Player { str name; int x; int lives; }

Player p = Player{name: "ada", x: 1, lives: 3};
p.x += 2;
println(p); // Player{name: "ada", x: 3, lives: 3}
```

Structs, lists and text are passed by reference: a function that gets a `Player` changes that same `Player`.

FILES

```
File f = open("notes.txt", "w");
write(f, "a line\n");
close(f);
str all = read(open("notes.txt", "r"));
```

```
println("count is ", count);
printf("%s: %d at %Zf\n", name, count, price);
// whole numbers
float price = 2.50; // with a point
bool done = false; // true or false
str name = "Ada"; // text
const int SIZE = 10; // never changes
```

Every variable has a type, and says it:

VALUES AND VARIABLES

Nothing changes type by itself (except an `int` where a `float` is wanted):

```
str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too
SUMS
+ - * / % as usual; x += 2, x++; 7 / 2 is 3.70 / 2 is 3.5.
random(6) is 0 to 5, sqrt, pow, sin, abs, min, max.
```

Functions and lists

```
int square(int n) {
    return n * n;
}

void greet(str who, int times) {
    for (int i = 0; i < times; i++)
        println("hi ", who);
}
```

A function can be used above where it is written. One that returns something must return on every path: the compiler checks.

LISTS (ARRAYS)

```
int[] scores = [12, 7, 30];
push(scores, 18); // [12, 7, 30, 18]
sort(scores); // [7, 12, 18, 30]
int best = scores[len(scores) - 1];
int last = pop(scores); // takes it off
int[][] grid = [[1, 2], [3, 4]];
```

Also `insert`, `remove_at`, `slice`, `reverse`, `find`. A wrong index stops the program and says where.

Deciding and repeating

A condition is a bool: `if (n != 0), never if (n).&& and, ||` or, `! not`.

```
if (count > 5) {
    println("many");
} else if (count > 0) {
    println("some");
} else {
    println("none");
}
while (lives > 0) { ... }
for (int i = 0; i < 10; i++) { ... }
```

EACH ITEM OF A LIST

```
str[] fruit = ["apple", "fig", "kiwi"];
for (str f in fruit) println("I like ", f);
break leaves a loop, continue goes on with the next time
round.
for ... in cannot go past the end. For a text's letters: for
```

Choosing: switch and enum

When one value decides between many things:

```
switch (key) {
case 'q', KEY_ESC:
    running = false;
case KEY_UP:
    y--;
default:
    beeps++;
}
```

Only the case that matches runs — nothing falls through into the next one, as it does in C. A case can list several values; they can be numbers, text or enum values. `default` comes last.

ENUM

Names for a handful of choices:

```
enum Dir { UP, DOWN, LEFT, RIGHT }

Dir d = LEFT;
switch (d) {
case UP, DOWN: println("up or down");
default: println("sideways");
}
```

Enum values are ints: UP is 0, DOWN 1, and LEFT = 10 sets one.

ANY COMMAND

```
output("date") runs a command and gives back what it
printed; run("play", "x.mp3") runs one and lets it print.
```

```
str[] songs = split(output("ls", "/home/user/music")
```

```
$ pico ~/pico/weather.pico Roma
25.4 C, clear
humidity 50%, wind 11.9 km/h
```

A path is names and numbers joined by dots: for {"list": [{"name": "a"}, {"name": "b"}]} counts the list.

```
str j = http_get("https://api.open-meteo.com/?v1/forecast?latitude=41.9&longitude=12.5"
&current_temperature_2m");
println(json_get(j, "current_temperature_2m"));
```

The internet

With Wi-Fi on, http_get(url) fetches what an address answers, as text ("") if it fails). Web APIs answer in JSON, and json_get(text, path) picks one value out of it.

When it goes wrong

The compiler says where, and often what to do:

```
hello.pico:4:13: error: expected int, got str;
parse it with to_int(s)
    int n = "12";
           ^
```

A mistake while it runs stops the program with the line, and how it got there; nothing else is harmed:

```
list.pico:6: runtime error: index 9 out of
range (length 3)
    in main (list.pico:6)
```

FINDING MISTAKES

<code>println(x)</code>	show a value; lists and structs too
<code>assert(ok, "why")</code>	stop at once if something is wrong
<code>Fn C</code>	stop a program that runs away
<code>picoc -d x.pico</code>	the compiled instructions

TIPS

Start from a program in ~/pico and change it. Keep functions short. Test on the PC too: make -C lang builds the same pico there. docs/PICO-TUTORIAL.md and docs/LANGUAGE.md have the rest.

The trick of any fast screen: build the frame as one text, write it once, and write only the cells that change.

life.pico (Conway's life) and mandel.pico (a Mandelbrot zoom) are next to it.

```
struct Drop { int y; int len; int speed; }
for (int x = 0; x < cols; x++) {
    str frame = "";
    Drop d = drops[x];
    for (int y = 0; y < d.len; y++) {
        frame += at(d.y, x) + HEAD + glyph();
        frame += at(d.y - 1, x) + GREEN + glyph();
        frame += at(d.y, x) + " " + " ";
        drops[x] = new_drop();
        if (readkey(50) == 'q') break;
    }
    print(frame);
}
```

Digital rain

pico ~/pico/matrix.pico: a drop for each column falls, a bright head writing new glyphs, a trail fading behind it. In outline:

Reference

```
int float bool str T[] File void
struct S { int a; } enum E { A, B }
const int N = 3; int[] a = [1, 2];
if else while for (;;) for (T x in a)
switch case default break continue return
?: && || ! + - * / % == != < > =
```

<code>print println printf format</code>	<code>random seed PI</code>
<code>len of text, list</code>	<code>open close read write</code>
<code>push pop insert remove_at</code>	<code>readLine exists is_dir</code>
<code>sort reverse slice find</code>	<code>listdir mkdir remove</code>
<code>split join trim chars</code>	<code>rename stdin stdout</code>
<code>substr replace repeat</code>	<code>run output getenv exit</code>
<code>upper lower contains</code>	<code>http_get json_get</code>
<code>starts_with ends_with</code>	<code>sleep_ms uptime_ms</code>
<code>to_int to_float str</code>	<code>time date the clock</code>
<code>chr ord codes</code>	<code>raw_mode readkey</code>
<code>abs min max sqrt pow</code>	<code>term_cols term_rows</code>
<code>sin cos tan atan2 hypot</code>	<code>assert checks</code>
<code>floor ceil round exp ln</code>	<code>KEY_UP ... KEY_ESC</code>

pico f.pico runs, picoc f.pico compiles, man pico reminds.

Steering is a switch over the keys, and an enum names the way it goes:

```
while (lover) {
    int key = readkey(delay);
    dir = steer(key, dir);
    Point next = ahead(snake[0], dir);
    if (hits(next)) over = true;
    insert(snake, 0, next); // the new head
    if (at(next, food)) grow();
    else erase(pop(snake)); // the tail
}
```

pico ~/pico/snake.pico: the whole game is a loop that reads a key, moves, and draws only what changed. In outline:



A game: snake

Programs to keep

In ~/pico on the machine, to run, read and take apart:

<code>matrix.pico</code>	digital rain, falling glyphs
<code>snake.pico</code>	the game: arrows or WASD
<code>weather.pico</code>	pico weather.pico Tokyo
<code>life.pico</code>	Conway's game of life
<code>mandel.pico</code>	a Mandelbrot zoom, in colour
<code>todo.pico</code>	a to-do list kept in a file
<code>guess.pico</code>	guess the number
<code>wc.pico</code>	counts lines, words, bytes
<code>hello.pico</code>	structs, arrays and a file
<code>benchmark.pico</code>	how fast the machine is

IDEAS TO TRY

A clock in big digits. A dice roller. Flash cards from a file in ~/notes. The day's news headlines from a JSON feed. A pomodoro timer that runs beep with run(). A maze. A calculator.

WHERE NEXT

man pico on the machine; on the PC, docs/PICO-TUTORIAL.md to learn and docs/LANGUAGE.md for all of it: every built-in, the limits, how the compiler works.

Run a program by its name. picoc snake.pico, put snake in ~/bin, and snake is a command.

```
int x = 10;
raw_mode(true);
print("\n[2J\n[25T");
for (;;) {
    printf("\n[10;%dH 0 ", x);
    int key = readkey(100);
    if (key == 'q') break;
    if (key == KEY_LEFT) x--;
    if (key == KEY_RIGHT) x++;
    raw_mode(false);
}
```

regardless.

gives KEY_NONE if nothing came, so the game moves on raw_mode(true). readkey(100) waits at most 100 ms and A game needs each key the moment it's pressed:

KEYS, AT ONCE

```
\[31m \[32m red, green text (30 to 37)
\[1m \[0m bold; back to normal
\[2J clear the screen
\[5;10H go to row 5, column 10
\[25T \[25h hide, show the cursor
```

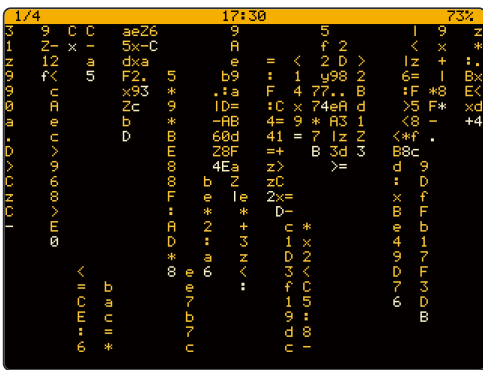
term_rows(). Colours and the cursor are escape codes, which are plain text:

The screen is 53 columns by 23 rows (term_cols()),

The terminal

PART 2 OF 2 · APPS

pico.
the language of PocketType



Part 2 colours and keys, a game loop, digital rain, the internet, commands, mistakes, and a reference on the back. All the programs are in ~/pico.

1.0 beta 4